TG4 – Modelos de interação com aplicação de padrões GRASP

Site de streaming personalizado

1. Caso de uso considerado

Foi usado o caso de uso principal/crítico do sistema, já definido na TG1: UC01 – Visualizar catálogo personalizado. Nele, o usuário acessa a plataforma e vê a home organizada dinamicamente em fileiras por gênero, calculadas a partir do seu histórico e de suas preferências recentes.

2. Diagrama de sequência de projeto

Ver arquivo separado: diagrama_sequencia_catalogo.svg (ou diagrama_sequencia_catalogo.puml para a versão em notação UML).

Participantes do diagrama: Usuário (ator), :TelaHome (fronteira), :CtrlHome (controlador), :usuarioLogado, :sessao, :catalogo e :conteudo (objetos de domínio) e :motorRecomendacao (classe de fabricação).

3. Padrões GRASP aplicados e motivo da utilização

Padrão GRASP	Onde foi aplicado	Responsabilidade atribuída
Controller	Mensagens 1-2 (:TelaHome → :CtrlHome)	CtrlHome recebe o evento de sistema "acessarHome()" vindo da interface e coordena todo o caso de uso, evitando que a lógica fique na própria tela.
Information Expert	Mensagem 3 (:CtrlHome → :usuarioLogado) e mensagens 7-8 (:catalogo → :conteudo)	usuarioLogado é quem sabe seus próprios gêneros favoritos; catalogo é quem sabe quais conteúdos armazena; conteudo é quem sabe seu próprio gênero. Cada consulta foi direcionada a quem detém a informação.
Creator	Mensagens 4-5 (:usuarioLogado cria :sessao)	Como Usuário "realiza" SessãoDeVisualização no modelo de domínio (associação 1 / 0..*), é o próprio usuarioLogado quem cria a nova sessão, e não o controlador.
Pure Fabrication (Fabricação)	Classe :motorRecomendacao	Calcular recomendações não é responsabilidade de nenhuma classe conceitual do domínio (não está no modelo de domínio). Para não sobrecarregar Catalogo nem usuarioLogado com essa lógica, foi criada uma classe artificial dedicada só a isso.
High Cohesion / Low Coupling	Separação entre CtrlHome, motorRecomendacao e catalogo	Cada classe ficou com um conjunto pequeno e focado de responsabilidades (coesão alta) e se comunica apenas com quem precisa (acoplamento baixo) — é a mesma motivação do exemplo Registradora/Venda visto em aula: evitar uma classe sobrecarregada concentrando tudo.
Indirection	motorRecomendacao entre CtrlHome e catalogo	motorRecomendacao atua como intermediário entre o controlador e o catálogo, evitando que CtrlHome dependa diretamente da lógica de armazenamento de conteúdos.

4. Discussão

O ponto de decisão mais importante do projeto foi onde colocar o cálculo da recomendação. Pelo Especialista na Informação, à primeira vista essa responsabilidade poderia ir para Catalogo (afinal, é ele quem guarda os conteúdos). Mas, assim como no exemplo da Registradora visto em aula, concentrar ali a lógica de recomendação deixaria Catalogo sobrecarregado e pouco coeso, além de acoplá-lo a um algoritmo que pode mudar com frequência (ex.: trocar o critério de pontuação, usar outro modelo preditivo).

Por isso, optou-se por uma Fabricação Pura (motorRecomendacao): uma classe sem correspondente no modelo de domínio, cuja única responsabilidade é calcular as fileiras recomendadas. Isso mantém Catalogo coeso (só armazena/consulta conteúdos) e isola o ponto de maior variação do sistema — o próprio critério de recomendação — atrás de uma interface estável, o que também se conecta ao princípio de Variações Protegidas: se o algoritmo mudar amanhã (outro modelo de recomendação, outro provedor), só motorRecomendacao precisa ser alterado, sem afetar CtrlHome, TelaHome ou Catalogo.

O Controlador (CtrlHome) foi escolhido em vez de deixar a lógica na própria tela porque TelaHome varia por dispositivo (celular, PC, TV) — exigido pelo critério de qualidade "consistência entre telas" definido na TG1. Mantendo toda a coordenação do caso de uso em CtrlHome, cada versão de TelaHome fica responsável só pela apresentação, podendo até usar polimorfismo (TelaHomeCelular, TelaHomeTV, TelaHomePC implementando uma interface comum) sem duplicar a lógica de negócio.